CS336 作业 1（基础）：构建 Transformer 语言模型

版本 26.0.3 · CS336 课程组 · 2026 年春季

译文说明：本文件完整翻译原作业说明，保留章节、题目标识、分值、实验要求与参考文献。公式、代码及图表按原 PDF 保留，并在需要时附中文说明；代码中的标识符不作翻译。中文页码与原文不一一对应，插图下方注明原文页码。本文件不包含习题解答。

1 作业概览

本作业要求从零构建训练标准 Transformer 语言模型（LM）所需的全部组件，并训练若干模型。

你将实现：①字节对编码（BPE）分词器（第 2 节）；②Transformer 语言模型（第 3 节）；③交叉熵损失函数与 AdamW 优化器（第 4 节）；④训练循环，支持序列化和加载模型及优化器状态（第 5 节）。

你将运行：①在 TinyStories 上训练 BPE 分词器；②用训练好的分词器将数据集编码为整数 ID 序列；③在 TinyStories 上训练 Transformer LM；④用训练好的模型生成样本并评估困惑度；⑤在 OpenWebText 上训练模型，将得到的困惑度提交到排行榜。

允许使用的工具。每个组件都应从头构建。尤其不得使用 torch.nn、torch.nn.functional 或 torch.optim 中的定义，以下例外除外：torch.nn.Parameter；torch.nn 中的容器类，例如 Module、ModuleList、Sequential；torch.optim.Optimizer 基类。

可使用其他任何 PyTorch 定义。若不确定某函数或类是否允许使用，可在 Slack 上询问。拿不准时，请考虑它是否违背“从零实现”的精神。容器完整列表：https://pytorch.org/docs/stable/nn.html#containers 。

关于 AI 工具。AI 可以完全自主地解决作业中的许多部分，但这会妨碍你深入参与课程内容并从中学习。允许使用 AI 回答高层次概念问题，或提供函数签名、库 API 等底层编程文档；不允许用 AI 实现任何作业的任何部分。这既包括 Cursor Agents、Codex、Claude Code 等编码智能体，也包括 Cursor Tab、GitHub Copilot 等 AI 自动补全。使用智能体时，确保它遵守提供的 AGENTS.md；使用聊天机器人时，也应提供相应提示词。

强烈建议做作业时关闭 IDE 中的 AI 自动补全。非 AI 自动补全，例如补齐函数名，完全可以使用。往届学生反馈，关闭 AI 补全有助于更深入地学习。完整政策：https://docs.google.com/document/d/1SZAlExB1qAc9izHt54gwunNpjKE6wXb8Y7yA_e-baK8/edit?tab=t.0 。

代码结构。作业代码与讲义位于 https://github.com/stanford-cs336/assignment1-basics 。请 git clone 克隆仓库；收到更新通知后，用 git pull 获取最新版。

1. cs336_basics/*：编写代码的目录，起初没有实现，可以从零自由构建。

2. adapters.py：规定实现必须具备的功能。对每项功能，例如缩放点积注意力，在对应接口 run_scaled_dot_product_attention 中调用你的代码。适配器只负责连接代码，不应包含实质性逻辑。

3. test_*.py：必须通过的测试，例如 test_scaled_dot_product_attention；它们调用 adapters.py 中的接口。不要修改测试文件。

提交方式。运行 make_submission.sh 创建提交压缩包。如果有大型数据或检查点不应打包，请将它们加入脚本的排除列表。向 Gradescope 提交：writeup.pdf，排版回答所有书面问题；code.zip，包含你编写的全部代码。

排行榜提交方式：向 https://github.com/stanford-cs336/assignment1-basics-leaderboard 提交 PR。详细说明见该仓库的 README.md。

数据集来源。使用预处理后的 TinyStories [R. Eldan 等，2023] 与 OpenWebText [A. Gokaslan 等，2019]，两者都是单个大型纯文本文件。正式选课学生可按计算资源指南下载；自学者可使用 README.md 中的命令。

资源有限时的建议：入门。课程讲义会用蓝色提示框介绍如何在较少甚至没有 GPU 的条件下完成部分作业，例如缩小数据集或模型，或在 Mac 集成 GPU、CPU 上训练。即使有课程机器可用，这些建议也能帮助加快迭代、节省时间，建议阅读。

资源有限时的建议：在 Apple Silicon 或 CPU 上完成作业 1。课程组参考实现可在配有 36 GB 内存的 Apple M4 Max 上训练出能生成较流畅文本的模型：Metal GPU（MPS）不到 5 分钟，CPU 约 30 分钟。不熟悉这些术语也没关系：只要笔记本较新、实现正确且高效，就能训练一个较流畅地生成简单儿童故事的小语言模型。后文会说明 CPU 或 MPS 所需的调整。

2 字节对编码（BPE）分词器

首先训练并实现字节级 BPE 分词器 [R. Sennrich 等，2016；C. Wang 等，2019]：将任意 Unicode 字符串表示为字节序列，在其上训练 BPE，再用分词器把文本字符串编码成整数 token 序列，供语言建模使用。

2.1 Unicode 标准

Unicode 是将字符映射到整数码点的文本编码标准。截至 2025 年 9 月发布的 Unicode 17.0，它定义了 172 种文字系统中的 159,801 个字符。例如“s”的码点是 115，通常写成 U+0073，其中 U+ 是惯用前缀，0073 是 115 的十六进制形式；“牛”的码点为 29275。Python 的 ord() 将单个 Unicode 字符转换为整数码点，chr() 将整数码点转为对应字符的字符串：

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

原文第 3 页

题目（unicode1）：理解 Unicode（1 分）

(a) chr(0) 返回哪个 Unicode 字符？

交付内容：一句话回答。

(b) 该字符的字符串表示（__repr__()）与打印出来的表示有什么不同？

交付内容：一句话回答。

(c) 这个字符出现在文本中时会怎样？可以在 Python 解释器中尝试以下操作，看结果是否符合预期：

```
>>> chr(0)
>>> print(chr(0))
```

原文第 3 页

```
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

原文第 4 页

交付内容：一句话回答。

2.2 Unicode 编码

直接在 Unicode 码点上训练分词器并不实用：词表过大，约 15 万项，而且许多字符很少出现，数据十分稀疏。因此改用将字符转为字节序列的 Unicode 编码。标准定义了 UTF-8、UTF-16 和 UTF-32 三种编码；UTF-8 是互联网的主流，超过 98% 的网页使用它。

Python 的 encode() 将 Unicode 字符串编码为 UTF-8；遍历 bytes 对象，例如调用 list()，可取得底层字节值；decode() 则将 UTF-8 字节串还原为 Unicode 字符串：

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # Get the byte values for the encoded string (integers from 0 to 255).
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # One byte does not necessarily correspond to one Unicode character!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

原文第 4 页

示例说明：字节值是 0—255 的整数，一个字节不一定对应一个 Unicode 字符；上例有 13 个字符，但 UTF-8 编码后占 23 字节。

将码点转为字节序列，相当于将具有 159,801 个有效取值的 21 位整数序列，变为取值范围 0—255 的字节序列。大小为 256 的字节词表更容易处理。字节级分词无需担心词表外 token，因为任意输入文本都能表示为 0—255 的整数序列。

题目（unicode2）：Unicode 编码（3 分）

(a) 相比 UTF-16 或 UTF-32，为什么更适合在 UTF-8 字节上训练分词器？可比较不同输入在这几种编码下的输出。

交付内容：1—2 句话回答。

(b) 下面的错误函数试图将 UTF-8 字节串解码为 Unicode 字符串。它为何不正确？给出一个会产生错误结果的输入字节串。

```
    def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
        return "".join([bytes([b]).decode("utf-8") for b in bytestring])

    >>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
    'hello'
```

原文第 5 页

交付内容：一个使 decode_utf8_bytes_to_str_wrong 输出错误的字节串，并用一句话解释原因。

(c) 给出一个无法解码成任何 Unicode 字符的双字节序列。

交付内容：一个例子及一句话解释。

2.3 子词分词

字节级分词缓解了词级分词的词表外问题，却会产生很长的输入序列，拖慢训练。例如，一个 10 词句子在词级模型中可能只有 10 个 token，在字符级模型中却可能达到 50 个甚至更多。长序列增加每一步的计算量，也引入更长距离的依赖，使字节序列上的语言建模更困难。

子词分词介于词级与字节级之间，用更大的词表换取更好的字节序列压缩。例如，原始文本中若经常出现 b'the'，给它一个独立词表项，就能将原来的 3 个 token 压缩成 1 个。

如何选择子词？R. Sennrich 等 [3] 提议使用字节对编码（BPE；P. Gage [5]），迭代地把出现最频繁的一对字节替换为一个新的、尚未使用的索引，即“合并”。新增子词旨在尽可能压缩输入；一个词若足够常见，就会成为单个子词单元。

用 BPE 构建词表的分词器通常称为 BPE 分词器。本作业实现字节级 BPE：词表项可以是单字节或合并后的字节序列，兼顾无词表外问题与可控序列长度。构建词表的过程称为“训练”分词器。

2.4 BPE 分词器训练

训练主要分为三步。

词表初始化。词表建立字节串 token 与整数 ID 的一一映射。字节级 BPE 的初始词表包含全部 256 种字节值，因此初始大小为 256。

预分词。原则上可以直接统计文本中相邻字节的频率，并从最高频的一对开始合并；但每次合并都扫描整个语料，计算代价很高。直接跨语料合并还可能产生仅标点不同的 token，例如 dog! 和 dog.；它们语义很接近，却会获得完全不同的 ID。

因此先对语料作粗粒度的预分词。例如预 token“text”出现 10 次，统计相邻的“t”“e”时可直接加 10，无需再次扫描全部出现位置。训练字节级 BPE 时，每个预 token 都表示为 UTF-8 字节序列。

R. Sennrich 等 [3] 最初按空白分割，即 s.split(" ")。基于 SentencePiece 的分词器仍可见这种做法，例如 Llama 1 和 2。现代分词器多采用 GPT-2 [A. Radford 等，2019] 的正则预分词。本作业使用从 https://github.com/openai/tiktoken/pull/234/files 获取的、写法稍简洁的原始正则：

```
>>> PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

原文第 6 页

可以交互地尝试分割文本，以理解它的行为：

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '-', 'tokenize']
```

原文第 6 页

代码中应使用 re.finditer，在构建“预 token → 次数”映射时，避免另外存储全部预分词结果。

计算 BPE 合并。对每个预 token 的 UTF-8 字节序列，迭代统计相邻对，找到最高频的 (A,B)，将其所有出现位置替换为新 token AB，并加入词表。最终词表大小等于初始大小（这里为 256）加上训练中执行的合并次数。出于效率考虑，不统计跨越预 token 边界的相邻对。

频率相同时，应确定性地选择字典序较大的对。例如 ("A","B")、("A","C")、("B","ZZ")、("BA","A") 并列最高频时，应合并 ("BA","A")：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

原文第 6 页

原始 BPE [3] 要加入词尾 token。本作业的字节级 BPE 不加入它，因为空白、标点也在词表内；显式表示这些字符后，学到的合并自然会反映词边界。

特殊 token。有些字符串用于表达文档边界等元数据，例如 <|endoftext|>。编码时通常希望它始终保留为一个 token，即单个整数 ID，而不被拆开，从而知道语言模型何时停止生成。必须把特殊 token 加入词表，为其分配固定 ID。

文献 [3] 的算法 1 给出了低效但直观的 BPE 训练实现，基本遵循上述步骤。作为入门练习，可以实现并测试它，检查理解是否正确。

示例（bpe_example）：BPE 训练

下面是改编自文献 [3] 的示例语料，词表另有特殊 token <|endoftext|>：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

原文第 7 页

初始化词表：特殊 token 加全部 256 种字节值。

为突出合并过程，假设预分词仅按空白分割，统计得到 {low:5, lower:2, widest:3, newest:6}。可表示为 dict[tuple[bytes,...],int]，例如 {(l,o,w):5,...}。注意，Python 中即使单个字节也是 bytes 对象；没有独立的 byte 类型，正如没有独立的 char 类型。

统计连续相邻对，并按单词频次累加，得到 {lo:7, ow:7, we:8, er:2, wi:3, id:3, de:3, es:9, st:9, ne:6, ew:6}。('e','s') 与 ('s','t') 并列，选择字典序较大的 ('s','t')。合并后为 {(l,o,w):5, (l,o,w,e,r):2, (w,i,d,e,st):3, (n,e,w,e,st):6}。

第二轮最高频为 (e,st)，次数 9，合并后为 {(l,o,w):5, (l,o,w,e,r):2, (w,i,d,est):3, (n,e,w,est):6}。继续进行，最终合并顺序为 ['s t','e st','o w','l ow','w est','n e','ne west','w i','wi d','wid est','low e','lowe r']。

若只做 6 次合并，则使用 ['s t','e st','o w','l ow','w est','n e']，词表为 [<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]。由此，newest 被分为 [ne,west]。

2.5 BPE 训练实验

在 TinyStories 上训练字节级 BPE。下载方式见第 1 节。开始前建议先查看数据内容。

并行化预分词。预分词通常是主要瓶颈，可用内置 multiprocessing 并行化。建议把语料分块，并使块边界位于特殊 token 的起始位置。可直接使用以下起始代码寻找边界，再把各块分给进程：https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py 。

这种分块始终有效，因为我们不希望跨文档合并。本作业可以始终如此分割，不必处理超大语料没有 <|endoftext|> 的边界情况。

预分词前移除特殊 token。运行正则 re.finditer 前，先从语料或分块中分离全部特殊 token。必须沿特殊 token 分割，不能让合并跨越它划定的边界。例如 [Doc 1]<|endoftext|>[Doc 2] 应分别对两篇文档预分词。特殊 token 是训练时的硬分段边界，但本身不参与合并计数。可使用 re.split，以 "|".join(special_tokens) 构造分隔模式；注意用 re.escape 转义，因为特殊 token 中可能包含 |。test_train_bpe_special_tokens 会检查这一点。

优化合并。朴素实现每次都遍历全部相邻对寻找最高频项，很慢。实际上，合并后只有与被合并对重叠的相邻对计数会变化。因此可以索引全部相邻对计数并增量更新，避免每轮重新统计。缓存可显著加速，但 BPE 的合并部分在 Python 中无法并行化。

资源有限时的建议：性能剖析。用 cProfile 或 py-spy 找到瓶颈，集中优化。

资源有限时的建议：缩小规模。不要直接从完整 TinyStories 开始，先用小“调试数据集”，例如 22K 篇文档的验证集，而不是 2.12M 篇文档的训练集。缩小数据、模型等规模是加快开发的一般策略。调试规模要足以暴露完整设置的同类瓶颈，使优化能迁移，又不能大到每次运行都耗时太久。

题目（train_bpe）：训练 BPE 分词器（15 分）

交付内容：编写函数，给定输入文本路径，训练字节级 BPE 分词器。至少支持以下输入：

• input_path: str：训练文本文件路径。

• vocab_size: int：正整数，规定最终词表的最大大小，包括初始字节、合并产生的 token 和特殊 token。

• special_tokens: list[str]：要加入词表的字符串。训练时作为硬边界，禁止跨越它们合并，但不纳入合并统计。

返回：vocab: dict[int,bytes]，从 token ID 映射到 token 字节串；merges: list[tuple[bytes,bytes]]，每项 (<token1>,<token2>) 表示这两个 token 被合并，按创建顺序排列。

实现 adapters.run_train_bpe，再运行 uv run pytest tests/test_train_bpe.py，应通过全部测试。

选做：可以用 C++（例如 cppyy、nanobind）或 Rust（PyO3）实现关键部分，但可能耗费大量时间。注意哪些操作复制数据，哪些直接读取 Python 内存；提供构建说明，或确保仅靠 pyproject.toml 就能构建。GPT-2 正则在多数正则引擎中支持不好，即使支持也常很慢。课程组确认 Oniguruma 支持负向先行断言且速度尚可，不过 Python 的 regex 包可能还更快。

题目（train_bpe_tinystories）：在 TinyStories 上训练 BPE（2 分）

(a) 训练最大词表为 10,000 的字节级 BPE，将 <|endoftext|> 加入词表。把词表与合并规则序列化到磁盘，进一步检查。训练耗时和内存是多少？最长 token 是什么，是否合理？资源要求：不使用 GPU，耗时 ≤30 分钟，内存 ≤30 GB。

提示：预分词使用多进程，并利用两点，应能在 2 分钟内完成训练：(a) 数据文件以 <|endoftext|> 分隔文档；(b) 应用 BPE 合并前已单独处理该特殊 token。

交付内容：1—2 句话回答。

(b) 对代码做性能剖析。训练流程哪部分最耗时？

交付内容：1—2 句话回答。

接下来在 OpenWebText 上训练；同样建议先查看数据。

题目（train_bpe_expts_owt）：在 OpenWebText 上训练 BPE（2 分）

(a) 训练最大词表为 32,000 的字节级 BPE，序列化保存词表与合并规则。最长 token 是什么，是否合理？资源要求：不使用 GPU，耗时 ≤12 小时，内存 ≤100 GB。

交付内容：1—2 句话回答。

(b) 比较在 TinyStories 与 OpenWebText 上训练出的分词器有何异同。

交付内容：1—2 句话回答。

2.6 BPE 分词器：编码与解码

前面已得到词表和合并列表，现在实现加载它们并在文本与 token ID 之间编码、解码的分词器。

2.6.1 文本编码

编码过程与训练对应。第 1 步，预分词，将每个预 token 表示为 UTF-8 字节序列，各自独立处理，不跨预 token 边界合并。第 2 步，按训练时的创建顺序，将合并规则应用于预 token。

示例（bpe_encoding）：BPE 编码

输入 'the cat ate'，词表为 {0:b' ',1:b'a',2:b'c',3:b'e',4:b'h',5:b't',6:b'th',7:b' c',8:b' a',9:b'the',10:b' at'}；合并列表为 [(b't',b'h'),(b' ',b'c'),(b' ',b'a'),(b'th',b'e'),(b' a',b't')]。

预分词结果为 ['the',' cat',' ate']。第一个预 token 起初为 [b't',b'h',b'e']，先应用首个可用规则 (b't',b'h')，得到 [b'th',b'e']；再找到下一个可用规则 (b'th',b'e')，得到 [b'the']。已合成一个 token，不能再合并，对应 ID 为 [9]。

' cat' 合并后为 [b' c',b'a',b't']，对应 [7,1,5]；' ate' 合并后为 [b' at',b'e']，对应 [10,3]。最终编码为 [9,7,1,5,10,3]。

特殊 token。编码时必须正确处理构造分词器时由用户指定的特殊 token。

内存考虑。若大文件无法装入内存，应将其或其他数据流拆成可控大小的块，依次处理，使内存复杂度保持常数，而非随文本大小线性增长。必须保证 token 不跨块边界，否则结果会与整段文本一次性编码不同。

2.6.2 文本解码

逐个查询 ID 对应的字节串，拼接后解码为 Unicode。用户可能提供任意整数 ID 序列，因此结果不保证是有效 Unicode。遇到非法字节时，应替换为官方替代字符 U+FFFD。bytes.decode 的 errors 参数控制错误处理，errors='replace' 会自动完成替换。详见 https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character 。

题目（tokenizer）：实现分词器（15 分）

交付内容：实现 Tokenizer 类，给定词表与合并列表，将文本编码为整数 ID，并将整数 ID 解码为文本。支持用户指定的特殊 token，尚不在词表中的应追加进去。建议接口：

def __init__(self, vocab, merges, special_tokens=None)：构造分词器。vocab: dict[int,bytes]；merges: list[tuple[bytes,bytes]]；special_tokens: list[str] | None = None。

def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)：类方法，从序列化的词表和合并列表构造并返回 Tokenizer，文件格式与训练代码输出一致。参数 vocab_filepath、merges_filepath 均为 str，special_tokens 为可选 list[str]。

def encode(self, text: str) -> list[int]：将输入文本编码为 token ID 序列。

def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]：接收字符串可迭代对象，例如文件句柄，返回惰性地产生 token ID 的生成器。大文件无法一次载入内存时，需依靠它高效分词。

def decode(self, ids: list[int]) -> str：将 token ID 序列解码为文本。

实现 adapters.get_tokenizer，运行 uv run pytest tests/test_tokenizer.py，应通过全部测试。

2.7 实验

题目（tokenizer_experiments）：分词器实验（4 分）

(a) 从 TinyStories 和 OpenWebText 各抽样 10 篇文档，分别用对应训练好的分词器（词表为 10K 和 32K）编码为整数 ID。各自的压缩率（字节/token）是多少？

交付内容：1—2 句话回答。

(b) 用 TinyStories 分词器处理 OpenWebText 样本会怎样？比较压缩率，或作定性描述。

交付内容：1—2 句话回答。

(c) 估算分词器吞吐量，例如字节/秒。处理 Pile 数据集的 825 GB 文本需要多久？

交付内容：1—2 句话回答。

(d) 用两个分词器分别编码各自的训练集和开发集，得到整数 token ID 序列，供后续训练语言模型。建议序列化为 uint16 类型的 NumPy 数组。为何 uint16 合适？

交付内容：1—2 句话回答。

3 Transformer 语言模型架构

语言模型输入一批整数 token ID，形状为 (batch_size,sequence_length)，输出各位置下一词在词表上的归一化概率分布，形状为 (batch_size,sequence_length,vocab_size)。训练时用预测分布与真实下一词计算交叉熵。推理生成时，取最后一个时间步的分布，选择下一 token，例如取最大概率项或随机采样，把它追加到输入，再重复。

本节从零构建 Transformer LM，先介绍总体结构，再逐一实现组件。

3.1 Transformer LM

将 token ID 经输入嵌入层转为稠密向量，经过 num_layers 个 Transformer 块，再通过可学习线性投影（“输出嵌入”或“LM head”）得到下一 token 的 logits。结构如图：

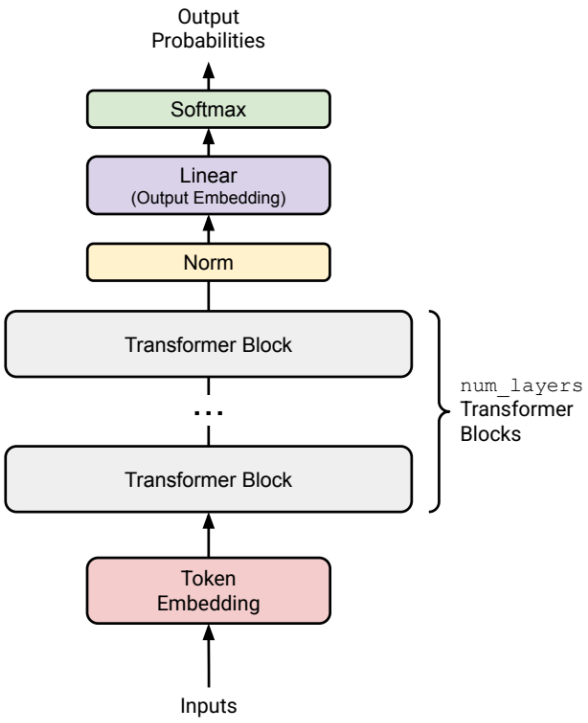


Figure 1: An overview of our Transformer language model.

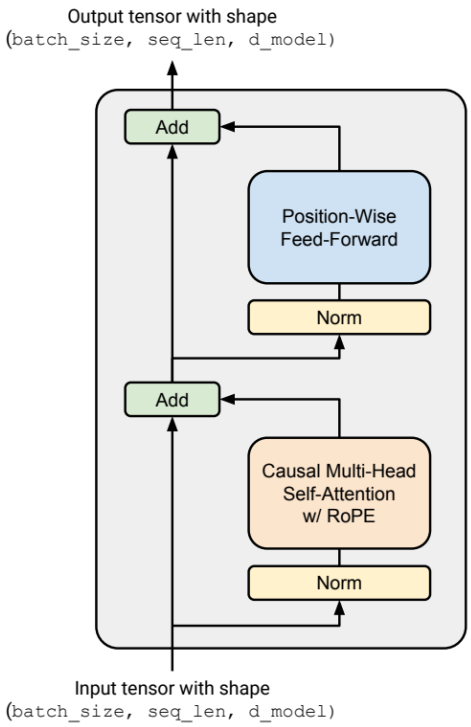


Figure 2: A pre-norm Transformer block.

原文第 13 页

图 1（左）：模型从下到上依次为输入、token 嵌入、num_layers 个 Transformer 块、归一化、线性输出嵌入、softmax、输出概率。图 2（右）：前置归一化块从输入开始，先归一化，经过带 RoPE 的因果多头自注意力，再与输入作残差相加；随后归一化、逐位置前馈网络，再残差相加得到输出。块的输入输出均为 (batch_size,seq_len,d_model)。

token 嵌入。第一步将批量 ID 序列转为包含 token 身份信息的向量序列，即图 1 的红色模块。嵌入层输入整数张量 (batch_size,sequence_length)，输出 (batch_size,sequence_length,d_model)。

前置归一化 Transformer 块。嵌入后的激活经过若干结构相同的层。标准仅解码器 Transformer 有 num_layers 个这样的块，各块输入输出形状均为 (batch_size,sequence_length,d_model)。自注意力聚合序列信息，前馈层作非线性变换。

最后将激活变为词表上的分布。本作业使用第 3.4 节详述的 pre-norm 块，因此在最后一个块之后还需归一化，保证输出尺度合适；再用可学习线性变换产生下一 token 的 logits，例如文献 [7] 的公式 2。

3.2 说明：批处理、Einsum 与高效计算

Transformer 中常对许多类似“批次”的输入重复相同计算：对批次中每个样本应用同样的前向过程；RMSNorm 与前馈网络对序列每个位置执行相同操作；多头注意力沿注意力头维并行执行注意力。

应以易读且充分利用 GPU 的方式实现。许多 PyTorch 运算允许张量前部附加批次维，并高效广播运算。例如 D 的形状为 (batch_size,sequence_length,d_model)，A 为 (d_model,d_model)，则 D @ A 是高效的批量矩阵乘法，前两维作为批次维。

因此，函数最好允许额外的前导批次维。为整理维度，可能需要多次 view、reshape、transpose，代码容易冗长且难以看清形状。

更方便的方式是 torch.einsum，或使用框架无关的 einops、einx。einsum 对任意输入维度作张量缩并；rearrange 对维度重排、合并与拆分。机器学习运算大多可以分解为维度变换、张量缩并，再加少量通常逐元素的非线性。因此 einsum 记法能让代码更易读、更灵活。

强烈建议学习和使用 einsum。不熟悉者先学 einops：https://einops.rocks/1-einops-basics/ ；熟悉 einops 者可学更一般的 einx：https://einx.readthedocs.io/en/stable/gettingstarted/basics.html 。两者已安装在提供的环境中。下面是对文档的补充示例，请先读文档。

说明：einops 生态较成熟，einx 尚未经同等程度的实践检验。若遇到限制或 bug，可退回 einops 加普通 PyTorch。

示例（einstein_example1）：用 einops.einsum 做批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#                          D                A     ->          Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

原文第 15 页

普通写法不容易看出 D、A 的形状和含义；einsum 显式写出输入输出轴，更便于理解。省略号版本允许 D 有任意前导维，同时约束 A 的维度。

示例（einstein_example2）：用 einops.rearrange 做广播运算

给定一批图像，希望按不同缩放系数，为每张图生成 10 个不同亮度的版本：

```
images = torch.randn(64, 128, 128, 3)  # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by,    "dim_value              -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

原文第 15 页

可以先重排再相乘，也可以直接用一个 einsum 完成。

示例（einstein_example3）：用 einops.rearrange 混合像素

输入图像形状为 (batch,height,width,channel)，希望对全部像素施加线性变换，各通道独立。变换矩阵 B 的形状为 (height*width,height*width)。先看普通写法：

```
channels_last = torch.randn(64, 32, 32, 3)  # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

原文第 16 页

改用 einops：

```
height = width = 32
## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

原文第 16 页

也可以用 einx.dot（对应 einops.einsum）一步完成：

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

原文第 16 页

普通实现可以在前后加注释说明形状，但繁琐且容易出错。einsum 记法让实现本身就是文档，既支持任意批次维，又明确输入输出形状。其他张量可考虑 Tensor 类型提示，例如不限于 JAX 的 jaxtyping：https://github.com/patrick-kidger/jaxtyping 。作业 2 会进一步讨论性能影响；目前只需知道，这些记法几乎总优于替代写法。

3.2.1 数学记号与内存顺序

许多机器学习论文采用行向量，与 NumPy 和 PyTorch 默认的行主序存储很契合。行向量的线性变换为：

$$y = xW^{\top}, \tag{1}$$

原文第 17 页

W∈R^(d_out×d_in) 按行主序存储，x∈R^(1×d_in) 为行向量。把 x 最外层维度增大，就可替换成 X∈R^(batch×d_in)，实现批处理。

线性代数更常用列向量，写作：

$$y = Wx, \tag{2}$$

原文第 17 页

W 形状相同，x∈R^d_in 为列向量。这时批次维需放最后，改用 X∈R^(d_in×batch)。

本作业数学表达主要使用列向量。但用 PyTorch 普通矩阵乘法实现时，应考虑其行主序习惯，像公式（1）那样转置矩阵；用 einsum 时，只需正确标注各轴。Matlab、Julia、Fortran 则采用列主序，批次维常放在最后；Python 相关库采用 C 的行主序惯例。

3.3 基础模块：Linear 与 Embedding

3.3.1 参数初始化

有效训练神经网络通常需要慎重初始化；不良初始化可能导致梯度消失或爆炸。pre-norm Transformer 对初始化相当稳健，但初始化仍显著影响速度与收敛。细节留待作业 3，本作业先采用通常有效的近似设置：

• 线性层权重：均值 0、方差 2/(d_in+d_out) 的正态分布，截断在 [−3σ,3σ]。

• 嵌入权重：均值 0、方差 1 的正态分布，截断在 [−3,3]。

• RMSNorm：全 1。

使用 torch.nn.init.trunc_normal_ 初始化截断正态权重。

3.3.2 线性模块

线性层是 Transformer 及一般神经网络的基础。实现继承 torch.nn.Module 的 Linear 类，执行：

$$y = Wx. \tag{3}$$

原文第 18 页

与多数现代 LLM 一样，不使用偏置项。

题目（linear）：实现线性模块（1 分）

交付内容：实现继承 torch.nn.Module 的 Linear 类，接口仿照 nn.Linear，但不包含 bias 参数或偏置权重。建议接口：

def __init__(self, in_features, out_features, device=None, dtype=None)：构造模块。in_features: int 为输入最后一维；out_features: int 为输出最后一维；device: torch.device | None = None 指定参数设备；dtype: torch.dtype | None = None 指定参数数据类型。

def forward(self, x: torch.Tensor) -> torch.Tensor：对输入施加线性变换。

必须继承 nn.Module、调用父类构造函数、将 W 而非 W^T 存为 nn.Parameter；不得使用 nn.Linear 或 nn.functional.linear。用上节设置及 torch.nn.init.trunc_normal_ 初始化。

实现 adapters.run_linear，让适配器把给定权重加载进模块，可用 Module.load_state_dict；然后运行 uv run pytest -k test_linear。

3.3.3 嵌入模块

Transformer 第一层将整数 token ID 映射到 d_model 维向量空间。实现继承 torch.nn.Module 的 Embedding 类，不使用 nn.Embedding。forward 接收形状 (batch_size,sequence_length) 的 torch.LongTensor，用 ID 索引形状 (vocab_size,d_model) 的嵌入矩阵，选出对应向量。

题目（embedding）：实现嵌入模块（1 分）

交付内容：实现继承 torch.nn.Module、执行嵌入查找的 Embedding 类，接口仿照 nn.Embedding。建议接口：

def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)：num_embeddings: int 为词表大小；embedding_dim: int 为嵌入维度 d_model；device: torch.device | None = None 为参数设备；dtype: torch.dtype | None = None 为参数类型。

def forward(self, token_ids: torch.Tensor) -> torch.Tensor：查找给定 ID 的嵌入向量。

必须继承 nn.Module、调用父类构造函数、将嵌入矩阵初始化为 nn.Parameter，并使 d_model 位于矩阵最后一维；不得使用 nn.Embedding 或 nn.functional.embedding。按前述设置用 torch.nn.init.trunc_normal_ 初始化。

实现 adapters.run_embedding，运行 uv run pytest -k test_embedding。

3.4 前置归一化 Transformer 块

每个块包含两个子层：多头自注意力和逐位置前馈网络 [8，第 3.1 节]。

原始 Transformer 在每个子层外加残差连接，再归一化，称为 post-norm。多项研究发现，将归一化移到子层输入端，并在最后一个块后补一次归一化，能改善训练稳定性 [T. Q. Nguyen 等，2019；R. Xiong 等，2020]，如图 2 的 pre-norm 结构。子层输出通过残差连接与子层输入相加 [8，第 5.4 节]。

直觉上，pre-norm 从输入嵌入到最终输出保留了一条不经过归一化的干净“残差流”，有利于梯度传播。它已成为 GPT-3、LLaMA、PaLM 等模型的常用结构，本作业也实现这一版本。下面逐个实现组件。

3.4.1 均方根层归一化

原始 Transformer 用 LayerNorm [J. L. Ba 等，2016] 归一化激活。遵循 LLaMA [12]，本作业使用均方根层归一化（RMSNorm；[13]，公式 4）。对激活向量 a∈R^d_model，每个分量按下式缩放：

$$\mathrm{RMSNorm}(a_i) = \frac{a_i}{\mathrm{RMS}(a)} g_i, \tag{4}$$

原文第 19 页

其中 RMS(a)=√[(1/d_model)Σ$_i$a$_i$²+ε]，g$_i$ 是可学习的增益参数，共 d_model 个；ε 常固定为 1e-5。

平方运算前应将输入升为 torch.float32，防止溢出。forward 的总体结构如下，返回前再转换回原始 dtype：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

原文第 20 页

题目（rmsnorm）：均方根层归一化（1 分）

交付内容：将 RMSNorm 实现为 torch.nn.Module。建议接口：

def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)：d_model 为模型隐藏维度，eps 为数值稳定常数，device 与 dtype 分别指定参数设备和类型。

def forward(self, x: torch.Tensor) -> torch.Tensor：处理形状 (batch_size,sequence_length,d_model) 的张量，返回相同形状。

注意，归一化前将输入升为 torch.float32，之后再转回原始类型。实现 adapters.run_rmsnorm，运行 uv run pytest -k test_rmsnorm。

3.4.2 逐位置前馈网络

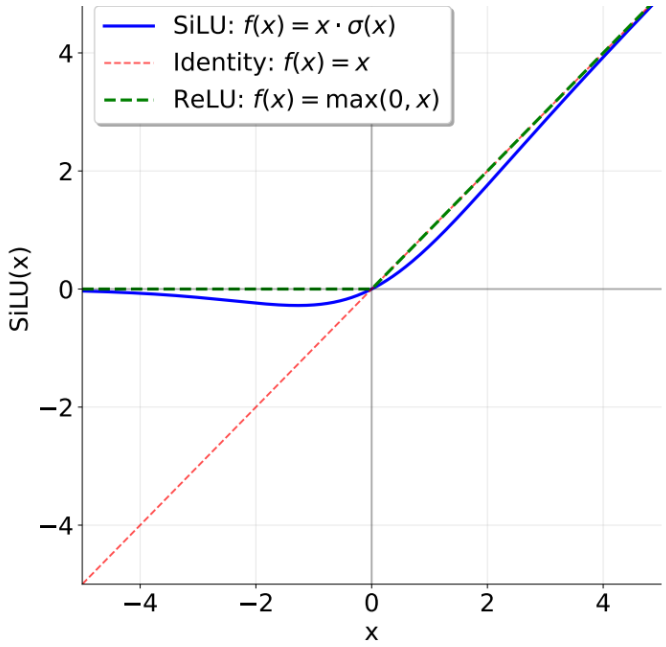


Figure 3: Comparing the SiLU (aka Swish) and ReLU activation functions.

原文第 21 页

图 3：比较 SiLU（也称 Swish）、ReLU 及恒等函数。SiLU 的形状接近 ReLU，但在零点平滑。

原始 Transformer 的前馈网络由两个线性变换及中间的 ReLU(x)=max(0,x) 构成，内部维度通常是输入维度的 4 倍 [8，第 3.3 节]。

现代模型通常改用其他激活函数并加入门控。本作业实现 Llama 3、Qwen 2.5 等采用的 SwiGLU：结合 SiLU（Swish）与门控线性单元（GLU）。与 PaLM、LLaMA 以来的多数现代 LLM 一样，不使用线性层偏置。

SiLU/Swish [D. Hendrycks 等，2016；S. Elfwing 等，2017] 定义为：

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{5}$$

原文第 21 页

GLU 最初由 Y. N. Dauphin 等 [19] 定义：将一个线性变换经 sigmoid 后，与另一个线性变换逐元素相乘：

$$\mathrm{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \tag{6}$$

原文第 21 页

⊙ 表示逐元素乘法。其设计动机是保留非线性能力，同时为梯度提供线性路径，缓解深层网络的梯度消失。

结合 SiLU 与 GLU，得到本作业前馈网络使用的 SwiGLU：

$$\mathrm{FFN}(x) = \mathrm{SwiGLU}(x, W_1, W_2, W_3) = W_2(\mathrm{SiLU}(W_1 x) \odot W_3 x), \tag{7}$$

原文第 22 页

x∈R^d_model，W₁、W₃∈R^(d_ff×d_model)，W₂∈R^(d_model×d_ff)，通常 d_ff=(8/3)d_model。具体实现可取附近的 64 倍数，提高硬件效率。

N. Shazeer [20] 首次提出这种结合，并发现它在语言建模上优于 ReLU 和无门控 SiLU 等基线。后文将比较 SwiGLU 与 SiLU。虽然这些设计有启发式解释，论文也有更多证据，但仍应保持实证态度。Shazeer 的一句名言是：“我们无法解释这些架构为什么似乎有效；我们把它们的成功，和其他一切一样，归功于上天的恩惠。”

题目（positionwise_feedforward）：实现逐位置前馈网络（2 分）

交付内容：实现由 SiLU 激活与 GLU 构成的 SwiGLU 前馈网络。本题允许使用 torch.sigmoid，以保证数值稳定。

令 d_ff 约为 (8/3)d_model，并保证内部维度是 64 的倍数，以充分利用硬件。实现 adapters.run_swiglu，再运行 uv run pytest -k test_swiglu。

3.4.3 相对位置嵌入

为注入位置信息，使用旋转位置嵌入（Rotary Position Embeddings，RoPE；J. Su 等，2021）。位置 i 的查询向量 q^(i)=W_q x^(i)∈R^d，经旋转矩阵 R^i 变为 q′^(i)=R^iq^(i)=R^iW_qx^(i)。

R^i 将相邻两个嵌入分量 q^(i)_(2k−1:2k) 视为二维向量，旋转角 θ_(i,k)=i/Θ^((2k−2)/d)，其中 k=1,...,d/2，Θ 为常数。因此 R^i 是 d×d 的分块对角矩阵，第 k 个块为：

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \tag{8}$$

原文第 22 页

完整矩阵为：

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \tag{9}$$

原文第 23 页

其中 0 表示 2×2 零矩阵。虽然可以构造完整矩阵，但好的实现应利用其结构更高效地计算。只关心序列内 token 的相对旋转，因此 cos、sin 值可以跨层、跨批次复用。

如需优化，可让所有层引用同一个 RoPE 模块，在初始化时用 self.register_buffer(persistent=False) 保存预计算的二维 sin/cos 缓冲区，而非 nn.Parameter，因为这些固定值不应学习。键向量 k^(j) 也要做同样的旋转，使用对应 R^j。本层没有可学习参数。

题目（rope）：实现 RoPE（2 分）

交付内容：实现 RotaryPositionalEmbedding 类，对输入应用 RoPE。建议接口：

def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)：构造模块并按需创建缓冲区。theta 是 Θ；d_k 为查询与键维度；max_seq_len 为最大输入序列长度；device 指定缓冲区设备。

def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor：输入形状 (...,seq_len,d_k)，输出相同形状，应允许任意数量批次维。token_positions 的形状为 (...,seq_len)，指定 x 沿序列维的 token 位置。

使用 token_positions 沿序列维索引可能预计算好的 cos/sin 张量。实现 adapters.run_rope，确保通过 uv run pytest -k test_rope。

3.4.4 缩放点积注意力

按原始 Transformer [8，第 3.2.1 节] 实现。先实现 softmax，将未归一化分数向量转为概率分布：

$$\mathrm{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^{n} \exp(v_j)}. \tag{10}$$

原文第 23 页

较大的 exp(v_i) 可能变成 inf，继而 inf/inf=NaN。softmax 对所有输入同时加同一个常数不变；利用这一点，通常先减去最大值，让最大输入变成 0，改善数值稳定性。

题目（softmax）：实现 softmax（1 分）

交付内容：函数接收一个张量及维度 i，沿该维执行 softmax，输出形状不变，维度 i 上形成归一化概率分布。计算时先沿该维减去最大值，避免数值问题。

实现 adapters.run_softmax，确保通过 uv run pytest -k test_softmax_matches_pytorch。

注意力定义为：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{11}$$

原文第 24 页

Q∈R^(n×d_k)，K∈R^(m×d_k)，V∈R^(m×d_v)。三者都是运算输入，不是可学习参数。

掩码。布尔掩码 M∈{True,False}^(n×m) 的第 i 行规定第 i 个查询可以关注哪些键。按这里的惯例，True 表示允许关注、信息可以流动，False 表示禁止。比如 [[True,True,False]] 让唯一的查询只关注前两个键。

使用掩码比对各子序列单独计算注意力高效。可在 softmax 前的 $QK^T/\sqrt{d_k}$ 上，为 False 位置加 $-\infty$。

题目（scaled_dot_product_attention）：实现缩放点积注意力（5 分）

交付内容：实现函数，处理形状 (batch_size,...,seq_len,d_k) 的查询与键，以及形状 (batch_size,...,seq_len,d_v) 的值；... 表示任意其他批次维。输出为 (batch_size,...,seq_len,d_v)，批次维说明见第 3.2 节。

支持可选的 (seq_len,seq_len) 布尔掩码；每个查询对应的 True 位置注意力概率总和应为 1，False 位置应为 0。

实现 adapters.run_scaled_dot_product_attention。uv run pytest -k test_scaled_dot_product_attention 测试三阶输入；uv run pytest -k test_4d_scaled_dot_product_attention 测试四阶输入。

3.4.5 因果多头自注意力

按 [8，第 3.2.2 节] 实现，多头注意力定义为：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h) \tag{12}$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \tag{13}$$

原文第 25 页

Q_i、K_i、V_i 是 Q、K、V 在嵌入维上的第 i 个切片，大小分别为 d_k、d_k、d_v，i=1,...,h；Attention 为上一节的缩放点积注意力。由此构成多头自注意力：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}\big(W_Q x, W_K x, W_V x\big) \tag{14}$$

原文第 25 页

可学习参数为 W_Q、W_K∈R^(hd_k×d_model)，W_V∈R^(hd_v×d_model)，W_O∈R^(d_model×hd_v)。因为多头操作会切分 Q、K、V，可将前三个权重理解为沿输出维按头分开。实现完成后，查询、键、值投影应总共只需 3 次矩阵乘法。进阶目标：将三种投影合到一个权重矩阵中，只做 1 次矩阵乘法。

因果掩码。模型不能关注未来 token。给定 $t_1,...,t_n$，预测前缀 $t_1,...,t_i$ 的下一词时，不应访问位置 i+1,...,n，因为推理时拿不到它们，而且它们会泄露真实下一词，使预训练任务失去意义。

朴素做法是对 n 个不同前缀分别运行自注意力；本作业改用因果掩码，使位置 i 只能关注 j≤i 的位置。可用 torch.triu 或广播的索引比较构造掩码，并复用上一节的掩码支持。

应用 RoPE。只旋转查询与键，不旋转值。注意力头维作为批次维，各头独立执行注意力，因此每个头的查询和键都使用相同的 RoPE 旋转。

题目（multihead_self_attention）：实现因果多头自注意力（5 分）

交付内容：实现 torch.nn.Module，至少接受 d_model: int（块输入维度）与 num_heads: int（注意力头数）。遵循 [8]，令 d_k=d_v=d_model/h。

实现 adapters.run_multihead_self_attention，运行 uv run pytest -k test_multihead_self_attention。

3.5 完整 Transformer LM

先组装图 2 的 Transformer 块。两子层分别为多头自注意力和 SwiGLU 前馈网络；各自先 RMSNorm，再主运算，最后残差相加。例如第一子层从 x 得到 y：

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \tag{15}$$

原文第 26 页

题目（transformer_block）：实现 Transformer 块（3 分）

按第 3.4 节和图 2 实现 pre-norm 块。至少接受 d_model: int（输入维度）、num_heads: int（头数）、d_ff: int（逐位置前馈内部维度）。

实现 adapters.run_transformer_block，运行 uv run pytest -k test_transformer_block。

交付内容：通过所提供测试的 Transformer 块代码。

随后按图 1 组装全模型：token 嵌入、num_layers 个块、最终归一化、LM head，得到词表上未归一化的分布，即 logits。嵌入见前文“Token 嵌入”部分，原文引用为第 3.1.0.1 节。

题目（transformer_lm）：实现 Transformer LM（3 分）

按第 3.1 节和图 1 组装模型。除 Transformer 块的全部构造参数外，至少接受：vocab_size: int，词表大小，决定嵌入矩阵维度；context_length: int，最大上下文长度，决定 RoPE sin/cos 缓冲区大小；num_layers: int，块数。

实现 adapters.run_transformer_lm，运行 uv run pytest -k test_transformer_lm。

交付内容：通过上述测试的 Transformer LM 模块。

资源核算。理解各模块的计算与内存开销很有用。Transformer 的绝大多数 FLOPs 来自矩阵乘法，因此基本方法是：列出前向传播的全部矩阵乘法，再换算其 FLOPs。

规则：A∈R^(m×n)、B∈R^(n×p)，则 AB 需要 2mnp FLOPs。因为每个输出元素是长度 n 的点积，按 n 次加法加 n 次乘法计为 2n FLOPs；共有 mp 个元素，总计 2mnp。做下一题前，可先逐个列出各组件的矩阵乘法及开销。

题目（transformer_accounting）：Transformer LM 资源核算（5 分）

(a) 使用本作业架构构建 GPT-2 XL 规模模型：vocab_size=50,257，context_length=1,024，num_layers=48，d_model=1,600，num_heads=25，d_ff=4,288（最接近 (8/3)×1,600 的 64 倍数）。共有多少可训练参数？若每个参数用单精度浮点表示，仅加载模型需多少内存？

交付内容：1—2 句话回答。

(b) 列出该模型前向传播所需矩阵乘法。输入长度为 context_length 时，总共需要多少 FLOPs？

交付内容：矩阵乘法列表及说明，以及总 FLOPs。

(c) 哪些部分消耗最多 FLOPs？

交付内容：1—2 句话回答。

(d) 对 GPT-2 small（12 层、d_model=768、12 头）、medium（24 层、1024、16 头）、large（36 层、1280、20 头）重复分析。模型变大时，各部分占总 FLOPs 的比例如何变化？

交付内容：逐模型列出各组件占前向总 FLOPs 的比例，并用 1—2 句话描述规模变化的影响。

(e) 将 GPT-2 XL 上下文长度增至 16,384。单次前向总 FLOPs 如何变化？各组件占比如何变化？

交付内容：1—2 句话回答。

4 训练 Transformer LM

已有分词预处理和模型，接下来补齐训练支持：定义交叉熵损失；实现用于最小化损失的 AdamW；构建加载数据、保存检查点、管理训练的循环。

4.1 交叉熵损失

对长度 m+1 的序列 x，模型在每个 i=1,...,m 定义 pθ(x_(i+1)|x_(1:i))。设训练集 D 中各序列均长 m+1，标准交叉熵，即负对数似然损失为：

$$\ell(\theta; D) = \frac{1}{|D|\ m} \sum_{x \in D} \sum_{i=1}^{m} -\log p_\theta(x_{i+1} \mid x_{1:i}). \tag{16}$$

原文第 28 页

一次前向传播即可得到全部位置的分布。模型为每个位置输出词表大小的 logits 向量 o_i，由此得到：

$$p(x_{i+1} \mid x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \tag{17}$$

原文第 28 页

$o_i[k]$ 表示向量第 k 项。交叉熵通常以 logits 和目标 x_(i+1) 定义，即目标上的狄拉克分布与预测 softmax(o_i) 的交叉熵。和 softmax 一样，实现时必须注意数值稳定性。

题目（cross_entropy）：实现交叉熵（1 分）

交付内容：函数接收预测 logits o_i 和目标 x_(i+1)，计算 $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$。要求：先减最大值以保证稳定；能抵消的 log 与 exp 尽量抵消；支持额外批次维，并返回批次上的平均损失。批次维总在词表维之前，见第 3.2 节。

实现 adapters.run_cross_entropy，运行 uv run pytest -k test_cross_entropy。

困惑度。训练用交叉熵已足够，评估时还要报告困惑度。长度 m 的序列，各位置损失为 $\ell_1,\ldots,\ell_m$，则：

$$\text{perplexity} = \exp\left(\frac{1}{m}\sum_{i=1}^{m} \ell_i\right). \tag{18}$$

原文第 29 页

4.2 SGD 优化器

最简单的基于梯度的优化器是随机梯度下降（SGD）。从随机初始化的 θ_0 出发，每步 t=0,...,T−1 更新：

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \tag{19}$$

原文第 29 页

B_t 为从 D 随机采样的批次，学习率 α_t 与批大小 $|B_t|$ 为超参数。

4.2.1 在 PyTorch 中实现 SGD

继承 torch.optim.Optimizer，需要实现两个方法。

def __init__(self, params, ...)：初始化。params 是待优化参数集合，或参数组，用于为模型不同部分设置不同学习率等超参数。将 params 传给父类构造函数，供 step 使用。优化器还可接收学习率等额外参数，并以字典形式传给父类，键为自行选定的超参数名称。

def step(self)：执行一次参数更新。训练循环在反向传播后调用，因此可以读取上一批次梯度。遍历参数张量 p，根据 p.grad（若存在）原地更新保存权重的 p.data。

API 有些细节，用例子更容易说明。下面实现带学习率衰减的 SGD，初始学习率 α，后续步长逐渐减小：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \tag{20}$$

原文第 30 页

对应的 PyTorch Optimizer 实现如下：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"]  # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p]  # Get state associated with p.
                t = state.get("t", 0)  # Get iteration number from the state, or 0.
                grad = p.grad.data  # Get the gradient of loss with respect to p.
                p.data -= lr / math.sqrt(t + 1) * grad  # Update weight tensor in-place.
                state["t"] = t + 1  # Increment iteration number.

        return loss
```

原文第 30 页

__init__ 将参数与默认超参数传给父类。如果传入的是单个 nn.Parameter 集合，父类会创建一个组并应用默认值；若已有参数组，则各组可有不同超参数。step 遍历组及组内参数，执行公式（20）。迭代次数作为每个参数的状态保存，先读取并用于更新，再递增。

API 允许用户提供 closure，在更新前重新计算损失。这里用不到，但为兼容 API 仍予以支持。下面是最小训练循环：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad()  # Reset the gradients for all learnable parameters.
    loss = (weights**2).mean() # Compute a scalar loss value.
    print(loss.cpu().item())
```

原文第 30 页

```
    loss.backward() # Run backward pass, which computes gradients.
    opt.step() # Run optimizer step.
```

原文第 31 页

每次迭代计算损失、执行优化器更新，这是训练循环的基本结构。训练语言模型时，参数来自模型，可用 m.parameters() 获取；损失在随机批次上计算，整体结构相同。

题目（learning_rate_tuning）：调节学习率（1 分）

学习率对训练影响很大。将上面 SGD 示例的学习率分别改为 1e1、1e2、1e3，各训练 10 次迭代。损失分别怎样变化：下降更快、更慢，还是发散，即随训练增大？

交付内容：用 1—2 句话描述观察结果。

4.3 AdamW

现代 LM 通常使用比 SGD 更复杂的优化器，多数衍生自 Adam [D. P. Kingma 等，2015]。本作业采用广泛使用的 AdamW [I. Loshchilov 等，2019]：通过与梯度更新解耦的权重衰减，每轮把参数向 0 拉近，以改善正则化。按文献 [23] 的算法 2 实现。

AdamW 为每个参数保存一阶、二阶矩的运行估计，以额外内存换取更好的稳定性和收敛。除学习率 α 外，还有控制矩估计更新的 β_1、β_2 及权重衰减率 λ。常用 $(\beta_1,\beta_2)=(0.9,0.999)$，而 LLaMA、GPT-3 等常用 (0.9,0.95)。ε 是防止 v 很小时数值不稳定的小常数，例如 1e-8。

Algorithm 1: AdamW Optimizer

Initialize learnable parameters θ

$m \leftarrow 0$ ▷ Initial value of the first moment vector; same shape as θ

$v \leftarrow 0$ ▷ Initial value of the second moment vector; same shape as θ

for $t = 1, \ldots, T$ **do**

  Sample batch of data B_t

  $g \leftarrow \nabla_\theta \ell(\theta; B_t)$ ▷ Compute the gradient of the loss

  $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$ ▷ Compute adjusted α for iteration t

  $\theta \leftarrow \theta - \alpha\lambda\theta$ ▷ Apply weight decay

  $m \leftarrow \beta_1 m + (1-\beta_1) g$ ▷ Update the first moment estimate

  $v \leftarrow \beta_2 v + (1-\beta_2) g^2$ ▷ Update the second moment estimate

  $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v}+\varepsilon}$ ▷ Apply moment-adjusted weight updates

end for

Algorithm 1: AdamW optimizer pseudocode.

原文第 32 页

算法 1 中文说明：初始化可学习参数 θ，令同形状的一阶矩 m、二阶矩 v 为零；从 t=1 开始，每次采样批次、计算损失梯度，按公式计算本步修正学习率 α_t；先施加权重衰减，再更新一阶、二阶矩，最后作矩估计修正后的权重更新。注意迭代计数从 1 开始。

题目（adamw）：实现 AdamW（2 分）

交付内容：实现 torch.optim.Optimizer 子类。__init__ 接收学习率 α，以及 β、ε、λ。父类提供 self.state，将每个 nn.Parameter 映射到状态字典，可保存 AdamW 的矩估计。

实现 adapters.get_adamw_cls，确保通过 uv run pytest -k test_adamw。

题目（adamw_accounting）：AdamW 训练资源核算（2 分）

假设所有张量均为 float32，计算运行 AdamW 的内存与计算需求。

(a) 峰值内存是多少？分解为参数、激活、梯度、优化器状态，并用 batch_size 及模型超参数 vocab_size、context_length、num_layers、d_model、num_heads 表示。假设 d_ff=(8/3)d_model。

为简化，激活只考虑：Transformer 块中的 RMSNorm；多头自注意力的 QKV 投影、QK^T、softmax、值的加权和、输出投影；SwiGLU 的 W_1、W_2、门控分支 SiLU、逐元素乘积、W_3；此外还有最终 RMSNorm、输出嵌入、logits 上的交叉熵。

交付内容：参数、激活、梯度、优化器状态各自的代数表达式及总和。

(b) 代入 GPT-2 XL 规模的模型，化为只依赖 batch_size 的表达式。80 GB 内存能容纳的最大批量是多少？

交付内容：形如 a·batch_size+b 的表达式，给出 a、b 数值，以及最大批大小。

(c) 一步 AdamW 需要多少 FLOPs？

交付内容：代数表达式及简要理由。

(d) 模型 FLOPs 利用率（MFU）衡量实测吞吐量（token/秒）相对硬件理论峰值 FLOP 吞吐量的利用程度 [A. Chowdhery 等，2022]。NVIDIA H100 的“float32”理论峰值为 495 teraFLOP/s，实际指 TensorFloat-32，原文称其实际上是“bfloat19”。假设达到 50% MFU，在单张 H100 上，以批量 1024 训练 GPT-2 XL 共 400K 步，需要多久？按 [25,26] 假设反向 FLOPs 为前向的两倍。TF32 说明：https://en.wikipedia.org/wiki/TensorFloat-32 。

交付内容：所需小时数及简要理由。

4.4 学习率调度

使损失下降最快的学习率通常随训练变化。Transformer 常在训练早期使用较大步长，随后逐渐衰减。本作业实现 LLaMA 使用的余弦退火。另有一些调度会重新升高学习率，即重启，以尝试越过局部极小值。

调度器接收当前步 t 及初始、最终学习率等参数，返回本步使用的学习率。最简单的是恒定函数。

带预热的余弦退火接收 t、最大学习率 α_max、最小学习率 α_min、预热迭代数 T_w、余弦退火结束迭代 T_c，定义如下：

$$\textbf{(Warm-up)}\ \text{If } t < T_w, \text{ then } \alpha_t = \frac{t}{T_w}\alpha_{\max}.$$

$$\textbf{(Cosine annealing)}\ \text{If } T_w \le t \le T_c, \text{ then } \alpha_t = \alpha_{\min} + \frac{1}{2}\left(1 + \cos\left(\frac{t-T_w}{T_c-T_w}\pi\right)\right)(\alpha_{\max} - \alpha_{\min}).$$

$$\textbf{(Post-annealing)}\ \text{If } t > T_c, \text{ then } \alpha_t = \alpha_{\min}.$$

原文第 33 页

中文说明：t<T_w 时线性预热；T_w≤t≤T_c 时按余弦从最大值降到最小值；t>T_c 后保持 α_min。

题目（learning_rate_schedule）：实现带预热的余弦学习率调度（1 分）

编写函数，接收 t、α_max、α_min、T_w、T_c，按上述调度返回 α_t。实现 adapters.get_lr_cosine_schedule，确保通过 uv run pytest -k test_get_lr_cosine_schedule。

4.5 梯度裁剪

某些样本会产生很大的梯度，使训练不稳定。梯度裁剪在每次反向传播后、优化器更新前，对梯度范数设置上限。设所有参数的梯度组成 g，计算其二范数 $||g||_2$。若小于最大值 M 则不变；否则乘以 $M/(||g||_2+\varepsilon)$，其中 ε 例如 1e-6，用于数值稳定。得到的范数略小于 M。

题目（gradient_clipping）：实现梯度裁剪（1 分）

编写函数，接收参数列表及最大二范数，原地修改各参数梯度。使用 PyTorch 默认的 ε=1e-6。实现 adapters.run_gradient_clipping，确保通过 uv run pytest -k test_gradient_clipping。

5 训练循环

现在将已分词数据、模型与优化器组合起来。

5.1 数据加载器

分词后的数据是单个序列 $x=(x_1,...,x_n)$。即使源数据由不同网页、代码文件等文档组成，通常也将其连接为一个序列，文档间加入 <|endoftext|> 等分隔符。

加载器产生批次流，每批包含 B 条长度 m 的输入及长度 m 的下一 token 标签。例如 B=1、m=3 时，$([x_2,x_3,x_4],[x_3,x_4,x_5])$ 是一个合法批次。

这种方式简化了训练：任意 $1\le i\le n-m$ 都能作为有效起点，易于采样；序列等长，无需填充，有助于提高硬件利用率和批量；采样也无需加载完整数据，适合无法全部放入内存的大数据集。

题目（data_loading）：实现数据加载（2 分）

交付内容：函数接收包含整数 token ID 的 NumPy 数组 x、batch_size、context_length 和 PyTorch 设备字符串（如 'cpu' 或 'cuda:0'），返回输入序列与对应下一 token 目标两个张量。两者均形状 (batch_size,context_length)，包含 ID，位于指定设备。

实现 adapters.run_get_batch，运行 uv run pytest -k test_get_batch。

资源有限时的建议：CPU 或 Apple Silicon 数据加载。数据和模型应放到同一正确设备；CPU 使用 'cpu'，Apple Silicon（M 系列）使用 'mps'。参考：https://docs.pytorch.org/docs/stable/mps.html ；https://docs.pytorch.org/docs/stable/notes/mps.html ；https://developer.apple.com/documentation/metalperformanceshaders 。

若数据集大到无法载入内存，可用 Unix 的 mmap，将磁盘文件映射到虚拟内存，访问时才按需加载。NumPy 提供 np.memmap；若数组通过 np.save 保存，也可用 np.load(...,mmap_mode='r')。得到的类数组对象会按需读取，仿佛整个数据集都在内存中。

训练采样时务必使用内存映射模式，并指定与文件一致的 dtype。建议明确检查映射后的内容是否正确，例如有无超过预期词表大小的值。

5.2 检查点

训练时需要保存模型，以便从超时、机器故障等中断恢复，也便于事后研究训练动态、比较不同阶段的生成结果。

检查点应包含恢复训练所需状态：至少模型权重；若优化器有状态，如 AdamW，还需保存矩估计；为恢复学习率调度，还需保存迭代次数。

nn.Module.state_dict() 返回可学习权重字典，load_state_dict() 恢复；torch.optim.Optimizer 同样提供这些方法。torch.save(obj,dest) 可将对象写到路径或类文件对象；obj 可是包含张量、整数等的字典。用 torch.load(src) 重新加载。

题目（checkpointing）：实现模型检查点（1 分）

实现两个函数。

def save_checkpoint(model, optimizer, iteration, out)：将模型、优化器、迭代次数全部状态保存到 out。可用两者的 state_dict 和 torch.save(obj,out)，obj 通常为字典，但只要能恢复，也可自定格式。参数类型：model: torch.nn.Module；optimizer: torch.optim.Optimizer；iteration: int；out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]。

def load_checkpoint(src, model, optimizer)：从路径或类文件对象 src 读取检查点，恢复模型与优化器，并返回保存的迭代次数。可用 torch.load 及两者的 load_state_dict。src 的类型同上面的 out，model 与 optimizer 类型同上。

实现 adapters.run_save_checkpoint 和 adapters.run_load_checkpoint，确保通过 uv run pytest -k test_checkpointing。

5.3 训练循环

把所有组件加入主训练脚本。建议方便地用命令行参数等方式启动不同超参数的运行，后面会多次这样做，研究不同选择对训练的影响。

题目（training_together）：整合训练流程（4 分）

交付内容：编写脚本，在用户提供的输入上训练模型。至少建议支持：配置并控制模型与优化器超参数；用 np.memmap 高效加载大型训练和验证集；将检查点保存到用户指定路径；定期记录训练与验证表现，可输出控制台或 Weights and Biases 等服务：https://wandb.ai 。

6 文本生成

训练能力之外，还需从模型生成文本。模型接收长度 sequence_length 的整数序列（也可带批次），每个位置对应词表上的下一 token 预测。接下来把它转为新序列的采样流程。

Softmax。按惯例，LM 返回最终线性层输出 logits，需用公式（10）的 softmax 归一化。

解码。提供前缀 token，即提示词，让模型预测下一 token 分布，然后采样下一输出。一步解码接收 x_(1...t)，按下式返回 x_(t+1)：

$$P(x_{t+1} = i \mid x_{1\ldots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \tag{21}$$

$$v = \mathrm{TransformerLM}(x_{1\ldots t})_t \in \mathbb{R}^{\mathrm{vocab_size}} \tag{22}$$

原文第 37 页

TransformerLM 输出 (sequence_length,vocab_size)，取最后一行，因为需要位置 t 的下一 token 预测。

反复从单步条件分布采样，将上次生成的 token 追加到下次输入，直到生成 <|endoftext|>，或达到用户指定的最大生成数，即得到基本解码器。

解码技巧。小模型有时生成质量很低，两个简单技巧可改善。首先是温度缩放，在 softmax 中加入 τ：

$$\mathrm{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\mathrm{vocab_size}} \exp(v_j/\tau)}. \tag{23}$$

原文第 37 页

当 τ→0，最大 logits 项占主导，softmax 趋向集中在该项的 one-hot 向量。

其次是核采样（nucleus sampling），也称 top-p，通过截去低概率 token 修改分布。设 q 是经过温度缩放 softmax 后的词表概率分布，则：

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases} \tag{24}$$

原文第 38 页

V(p) 是累计概率至少为 p 的最小索引集合。将 q 从大到小排序，依次选取概率最大的词表项，直到累计达到 p 即可。

题目（decoding）：解码（3 分）

交付内容：实现语言模型解码函数。建议支持：为用户提示词生成续文，直到 <|endoftext|>；控制最大生成 token 数；按指定温度缩放下一 token 分布；按用户阈值进行 top-p / 核采样 [A. Holtzman 等，2020]。

7 实验

现在把所有组件结合起来，在预训练数据上训练小型语言模型。

7.1 实验方式与交付要求

理解 Transformer 架构设计的最好方式，是亲自修改、运行；动手经验无可替代。因此要能够快速、一致地实验，并记录做过什么。

为加快实验，使用约 17M 总参数的小模型和简单的 TinyStories 数据集；为保证一致性，系统地消融组件、改变超参数；为留下记录，需提交实验日志及各次运行的学习曲线。

为绘制损失曲线，务必定期评估验证损失，同时记录步数和实际运行时间。Weights and Biases 等日志工具可能有帮助。

题目（experiment_log）：实验日志（3 分）

为训练和评估代码建立实验跟踪设施，记录实验及损失曲线，同时以梯度更新步数和实际运行时间为横轴。

交付内容：实验日志基础设施代码，以及记录本节后续各题所有尝试的实验日志文档。

7.2 TinyStories

先从简单的 TinyStories [1] 开始，模型训练快，也能看到有趣行为。获取方法见第 1 节，以下为一个样本。

示例（tinystories_example）：TinyStories 样本

> Once upon a time there was a little boy named Ben. Ben loved to explore the world around him. He saw many amazing things, like beautiful vases that were on display in a store. One day, Ben was walking through the store when he came across a very special vase. When Ben saw it he was amazed! He said, “Wow, that is a really amazing vase! Can I buy it?” The shopkeeper smiled and said, “Of course you can. You can take it home and show all your friends how amazing it is!” So Ben took the vase home and he was so proud of it! He called his friends over and showed them the amazing vase. All his friends thought the vase was beautiful and couldn’t believe how lucky Ben was. And that’s how Ben found an amazing vase in the store!

原文第 39 页

中文译文：从前有个名叫 Ben 的小男孩，他喜欢探索周围的世界。他见过许多奇妙事物，比如商店里陈列的漂亮花瓶。一天，他逛商店时发现了一个很特别的花瓶，惊叹道：“哇，这个花瓶真了不起！我能买它吗？”店主微笑着说：“当然可以。把它带回家，让所有朋友看看它有多棒！”Ben 把花瓶带回家，为它感到十分自豪。他叫来朋友，展示这个奇妙的花瓶。朋友们都觉得花瓶很美，不敢相信 Ben 如此幸运。这就是 Ben 在商店里发现奇妙花瓶的故事！

7.2.1 超参数调节

给出以下起始设置，其他超参数由你寻找合适值。

• 词表大小 10,000。常见词表为数万至数十万，可以改变它，观察词表与模型行为的变化。

• 上下文长度 256。TinyStories 可能无需长序列，后面的 OpenWebText 则值得尝试不同长度，观察单步耗时和最终困惑度。

• d_model=512，略小于许多小型 Transformer 论文采用的 768，以加快实验。

• d_ff=1344，约为 (8/3)d_model，且是 64 的倍数，有利于 GPU 性能。

• RoPE 的 Θ=10000。

• 4 层、16 个头。原文称此配置约有 17M 个非嵌入参数，是较小的 Transformer。

• 总处理 token 数为 327,680,000，即 batch_size×总步数×context_length 应大致等于这个值。

通过试验为学习率、预热、AdamW 的 β_1、β_2、ε 以及权重衰减寻找良好默认值。典型选择可参考 [22]。

译注：第 7.1 节原文称“约 17M 总参数”，此处原文称“约 17M 非嵌入参数”；译文分别保留原文表述，具体实现配置以上列出的超参数为准。

7.2.2 整合运行

准备训练好的 BPE 分词器，对训练集编码，使用已写好的循环训练。正确且高效的实现，在上述设置下用单张 B200 应约需 20—30 分钟。若明显更慢，检查数据加载、检查点保存、验证损失计算是否成为瓶颈，以及是否正确批量化。

7.2.3 调试模型架构的技巧

建议熟悉 IDE 自带调试器，例如 VSCode/Zed，比打印调试更省时。纯文本编辑器用户可用 ipdb。其他好习惯包括：

• 开发网络时，先尝试过拟合单个小批次。正确实现应能很快将训练损失降到接近零。

• 在不同组件设断点，检查中间张量形状是否符合预期。

• 监控激活、模型权重和梯度范数，检查是否爆炸或消失。

题目（learning_rate）：调节学习率（2 B200 小时，3 分）

学习率是最重要的超参数之一。基于已训练的基础模型，回答以下问题。

(a) 扫描多个学习率，报告最终损失；若优化器发散，则注明。

交付内容：多个学习率的学习曲线，并解释搜索策略。

交付内容：一个在 TinyStories 上逐 token 验证损失不超过 1.45 的模型。

资源有限时的建议：在 CPU 或 Apple Silicon 上少训练一些。若设备为 cpu 或 mps，可将总处理 token 数减为 40,000,000，已足以生成较流畅文本；目标验证损失可放宽到 2.00。

课程组在 36 GB 内存的 M4 Max 上，用调好的学习率、32×5000×256=40,960,000 token，CPU 用时 1 小时 22 分钟，MPS 用时 36 分钟；第 5000 步验证损失为 1.80。

额外建议：使用 N 步训练时，使余弦衰减恰在第 N 步达到最低学习率。MPS 不要像 CUDA 那样设置 torch.set_float32_matmul_precision('high') 来启用 TF32；课程组在 torch 2.9.0 上发现，这可能使 MPS 后端无声地使用有问题的内核，导致训练不稳定。

可用 torch.compile 做 JIT 编译来加速：CPU 使用 model = torch.compile(model)；MPS 使用 model = torch.compile(model, backend="aot_eager")，可对反向传播做一定优化。原文所用 torch 2.9.0 的 MPS 尚不支持 Inductor 编译。

(b) 一种经验说法是最佳学习率处于“稳定性边缘”。研究开始发散的学习率与最佳学习率之间的关系。

交付内容：逐步增大学习率的学习曲线，至少有一次发散运行，并分析它与收敛速度的关系。

接下来改变批量。更大的矩阵乘法可提高 GPU 效率，但批量是否总是越大越好？通过实验观察。

题目（batch_size_experiment）：改变批大小（1 B200 小时，1 分）

将批大小从 1 逐步增大到显存极限，中间至少测试几个值，包括 64、128 等典型大小。

交付内容：不同批量的学习曲线，必要时重新优化学习率。

交付内容：用几句话讨论批量及其对训练的影响。

有了解码器，就可以生成文本并检查质量。作为参考，输出至少应与下面的例子相当。

示例（ts_generate_example）：TinyStories 模型生成样本

> Once upon a time, there was a pretty girl named Lily. She loved to eat gum, especially the big black one. One day, Lily's mom asked her to help cook dinner. Lily was so excited! She loved to help her mom. Lily's mom made a big pot of soup for dinner. Lily was so happy and said, "Thank you, Mommy! I love you." She helped her mom pour the soup into a big bowl. After dinner, Lily's mom made some yummy soup. Lily loved it! She said, "Thank you, Mommy! This soup is so yummy!" Her mom smiled and said, "I'm glad you like it, Lily." They finished cooking and continued to cook together. The end.

原文第 41 页

中文译文：从前有个漂亮女孩叫 Lily。她喜欢吃口香糖，尤其是那种又大又黑的。一天，妈妈让她帮忙做晚饭，Lily 很兴奋，她喜欢帮助妈妈。妈妈煮了一大锅汤。Lily 开心地说："谢谢妈妈！我爱你。"她帮妈妈把汤倒进大碗里。晚饭后，妈妈又做了些好喝的汤。Lily 很喜欢，说："谢谢妈妈！这汤真好喝！"妈妈笑着说："你喜欢就好，Lily。"她们做完了饭，又继续一起做饭。故事结束。

资源有限时的建议：CPU 或 Apple Silicon 上生成文本。若使用只处理 40M token 的设置，生成结果仍应像英语，但不如上例流畅。例如：

> Once upon a time, there was a little girl named Sue. Sue had a tooth that she loved very much. It was his best head. One day, Sue went for a walk and met a ladybug! They became good friends and played on the path together.
>
> "Hey, Polly! Let's go out!" said Tim. Sue looked at the sky and saw that it was difficult to find a way to dance shining. She smiled and agreed to help the talking!"
>
> As Sue watched the sky moved, what it was. She

原文第 42 页

中文译文（保留原文的语义跳跃、指代混乱与未完句）：从前有个小女孩叫 Sue。Sue 有一颗她很喜欢的牙齿。那是他最好的头。一天，Sue 出门散步，遇到了一只瓢虫！她们成了好朋友，在小路上一起玩。"嘿，Polly！我们出去吧！"Tim 说。Sue 看着天空，发现很难找到一种跳舞发光的方法。她笑了，同意帮助说话！"当 Sue 看着天空移动，原来是什么。她

题目（generate）：生成文本（1 分）

使用解码器和训练检查点，报告模型生成文本。可能需要调温度、top-p 等参数，才能获得流畅输出。

交付内容：至少 256 个 token 的文本，或到首个 <|endoftext|> 为止；简评流畅性，并指出至少两个影响输出好坏的因素。

7.3 消融与架构修改

亲自修改并观察，是理解 Transformer 的最好方式。下面做几项简单实验。

消融 1：层归一化。常说它对训练稳定性至关重要。先大胆去掉 RMSNorm，看看会怎样。

题目（layer_norm_ablation）：去掉 RMSNorm 并训练（0.5 B200 小时，1 分）

移除 Transformer 中全部 RMSNorm。用此前最佳学习率会怎样？降低学习率能否恢复稳定？

交付内容：移除 RMSNorm 后的训练曲线，以及使用最佳学习率的曲线。

交付内容：用几句话讨论 RMSNorm 的影响。

再考察归一化的位置。pre-norm 块定义为：

$$z = x + \mathrm{MultiHeadSelfAttention}(\mathrm{RMSNorm}(x)) \tag{25}$$

$$y = z + \mathrm{FFN}(\mathrm{RMSNorm}(z)). \tag{26}$$

原文第 42 页

这是相对于原始 Transformer 少数达成较广泛共识的修改之一。原始架构采用 post-norm：

$$z = \mathrm{RMSNorm}(x + \mathrm{MultiHeadSelfAttention}(x)) \tag{27}$$

$$y = \mathrm{RMSNorm}(z + \mathrm{FFN}(z)). \tag{28}$$

原文第 43 页

现在改回 post-norm，看看会怎样。

题目（pre_norm_ablation）：实现 post-norm 并训练（0.5 B200 小时，1 分）

将 pre-norm 实现改成 post-norm，训练并观察结果。

交付内容：post-norm 的学习曲线，并与 pre-norm 比较。

层归一化显著影响 Transformer 行为，甚至放置位置也很重要。

消融 2：位置嵌入。将使用 RoPE 的基线，与完全不使用位置嵌入的 NoPE 比较。理论上，带因果掩码的仅解码器 Transformer 即使没有显式位置嵌入，也能推断相对或绝对位置信息 [Y.-H. H. Tsai 等，2019；A. Kazemnejad 等，2023]。现在通过实验检验。

题目（no_pos_emb）：实现 NoPE（0.5 B200 小时，1 分）

修改使用 RoPE 的模型，完全去掉位置嵌入信息，观察结果。

交付内容：比较 RoPE 与 NoPE 表现的学习曲线。

消融 3：SwiGLU 与 SiLU。遵循 [20]，比较带门控 SwiGLU 与仅使用 SiLU、不含 GLU 的前馈网络，以检验门控的作用：

$$\mathrm{FFN}_{\mathrm{SiLU}}(x) = W_2\ \mathrm{SiLU}(W_1 x). \tag{29}$$

原文第 43 页

SwiGLU 使用约 (8/3)d_model 的内部维度，并取 64 的倍数以利用 GPU Tensor Core。无门控 SiLU 基线应改为 d_ff=4d_model，使其参数量与有三个权重矩阵的 SwiGLU 大致相同；SiLU 版本只有两个。

题目（swiglu_ablation）：SwiGLU 与 SiLU（0.5 B200 小时，1 分）

交付内容：参数量大致匹配时，比较 SwiGLU 与 SiLU 前馈网络表现的学习曲线。

交付内容：用几句话讨论发现。

资源有限时的建议：GPU 资源较少的在线学习者可继续在 TinyStories 上测试修改。后续将转向更大、更嘈杂的 OpenWebText，并可选提交课程排行榜。在 OpenWebText 上训练到流畅需要较长时间，因此资源有限时建议继续在 TinyStories 上修改架构，用验证损失评估。

7.4 在 OpenWebText 上运行

现在转向更标准的网页预训练数据。课程提供了 OpenWebText 的一个小子集，同样是单个文本文件，获取方式见第 1 节。下面的文本更真实、复杂、多样；建议浏览训练集，了解网页抓取语料的样貌。

示例（owt_example）：OWT 样本

> Baseball Prospectus director of technology Harry Pavlidis took a risk when he hired Jonathan Judge.
>
> Pavlidis knew that, as Alan Schwarz wrote in The Numbers Game, “no corner of American culture is more precisely counted, more passionately quantified, than performances of baseball players.” With a few clicks here and there, you can find out that Noah Syndergaard’s fastball revolves more than 2,100 times per minute on its way to the plate, that Nelson Cruz had the game’s highest average exit velocity among qualified hitters in 2016 and myriad other tidbits that seem ripped from a video game or science fiction novel. The rising ocean of data has empowered an increasingly important actor in baseball’s culture: the analytical hobbyist.
>
> That empowerment comes with added scrutiny – on the measurements, but also on the people and publications behind them. With Baseball Prospectus, Pavlidis knew all about the backlash that accompanies quantitative imperfection. He also knew the site’s catching metrics needed to be reworked, and that it would take a learned mind – someone who could tackle complex statistical modeling problems – to complete the job.
>
> “He freaks us out.” Harry Pavlidis
>
> Pavlidis had a hunch that Judge “got it” based on the latter’s writing and their interaction at a site-sponsored ballpark event. […]

原文第 44 页

中文译文：Baseball Prospectus 的技术主管 Harry Pavlidis 在雇用 Jonathan Judge 时冒了一个险。

Pavlidis 知道，正如 Alan Schwarz 在《The Numbers Game》中写的：“美国文化中，没有哪个角落像棒球运动员的表现这样，被如此精确地统计、如此热情地量化。”只需点几下鼠标，你就能查到 Noah Syndergaard 的快速球飞向本垒时每分钟旋转超过 2100 圈，Nelson Cruz 在 2016 年符合资格的击球手中拥有最高平均击球初速，以及无数仿佛来自电子游戏或科幻小说的小知识。不断扩大的数据海洋，壮大了棒球文化中日益重要的一群人：业余分析爱好者。

这种赋权也带来更多审视，不仅针对测量结果，也针对其背后的人和出版物。Pavlidis 很清楚 Baseball Prospectus 会因量化上的不完善而受到反对。他也知道网站的捕手指标需要重做，且必须由知识扎实、能处理复杂统计建模问题的人来完成。

“他让我们惊叹不已。”——Harry Pavlidis。

根据 Judge 的文章，以及两人在网站主办的球场活动中的交流，Pavlidis 直觉认为他“懂行”。 [……]

注意：本实验可能需要重新调整学习率、批大小等超参数。

题目（main_experiment）：OWT 实验（2 B200 小时，2 分）

使用与 TinyStories 相同的模型架构及总训练迭代数，在 OpenWebText 上训练。表现如何？

交付内容：OpenWebText 学习曲线，描述损失与 TinyStories 的差异，并解释这些损失应如何理解。

交付内容：按 TinyStories 输出的相同格式报告生成文本。它有多流畅？为何模型和计算预算相同，输出质量仍更差？

7.5 自行修改与排行榜

完成到这里已经接近终点。现在尝试改进 Transformer，比较你的架构和超参数与其他同学的表现。

排行榜规则。仅有两项限制：①单次提交在 B200 上最多运行 45 分钟，使用 SLURM 或 Modal 时可在提交脚本中强制限制；②只能使用提供的 OpenWebText 训练集。除此之外，可以自由尝试。

原文建议参考的思路来源：Llama 3 [14]、Qwen 2.5 [15] 等开源 LLM；以及 https://github.com/KellerJordan/modded-nanogpt ，其中社区分享了许多小规模语言模型预训练的提速修改。

例如，自原始 Transformer 起就常用输入、输出嵌入权重共享，见 [8，第 3.4 节] 与 [16，第 2 节]。若尝试权重共享，可能需要降低嵌入 / LM head 初始化的标准差。

先在 OpenWebText 小子集或 TinyStories 上测试，再进行完整 45 分钟运行。注意，本排行榜中有效的修改未必能推广到更大规模预训练；课程的缩放定律单元会继续探讨。

题目（leaderboard）：排行榜（10 B200 小时，6 分）

按上述规则训练，以在 0.75 B200 小时内尽可能降低验证损失为目标。

交付内容：最终记录的验证损失、一张明确以实际运行时间为横轴且时长少于 45 分钟的学习曲线，以及所做修改的说明。排行榜提交至少应优于损失 5.0 的朴素基线。提交地址：https://github.com/stanford-cs336/assignment1-basics-leaderboard 。

参考文献

保留论文英文题名，便于检索。

[1] R. Eldan、Y. Li，TinyStories: How Small Can Language Models Be and Still Speak Coherent English? 2023.

[2] A. Gokaslan、V. Cohen、E. Pavlick、S. Tellex，OpenWebText corpus. 2019.

[3] R. Sennrich、B. Haddow、A. Birch，Neural Machine Translation of Rare Words with Subword Units. ACL, 2016.

[4] C. Wang、K. Cho、J. Gu，Neural Machine Translation with Byte-Level Subwords. 2019.

[5] P. Gage，A new algorithm for data compression. C Users Journal, 12(2):23–38, 1994 年 2 月。

[6] A. Radford、J. Wu、R. Child、D. Luan、D. Amodei、I. Sutskever，Language Models are Unsupervised Multitask Learners. 2019.

[7] A. Radford、K. Narasimhan、T. Salimans、I. Sutskever，Improving Language Understanding by Generative Pre-Training. 2018.

[8] A. Vaswani 等，Attention is All you Need. NeurIPS, 2017.

[9] T. Q. Nguyen、J. Salazar，Transformers without Tears: Improving the Normalization of Self-Attention. IWSWLT, 2019.

[10] R. Xiong 等，On Layer Normalization in the Transformer Architecture. ICML, 2020.

[11] J. L. Ba、J. R. Kiros、G. E. Hinton，Layer Normalization. 2016.

[12] H. Touvron 等，LLaMA: Open and Efficient Foundation Language Models. 2023.

[13] B. Zhang、R. Sennrich，Root Mean Square Layer Normalization. NeurIPS, 2019.

[14] A. Grattafiori 等，The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783

[15] A. Yang 等，Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

[16] A. Chowdhery 等，PaLM: Scaling Language Modeling with Pathways. 2022.

[17] D. Hendrycks、K. Gimpel，Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units. 2016.

[18] S. Elfwing、E. Uchibe、K. Doya，Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning. https://arxiv.org/abs/1702.03118

[19] Y. N. Dauphin、A. Fan、M. Auli、D. Grangier，Language Modeling with Gated Convolutional Networks. https://arxiv.org/abs/1612.08083

[20] N. Shazeer，GLU Variants Improve Transformer. 2020.

[21] J. Su、Y. Lu、S. Pan、B. Wen、Y. Liu，RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021.

[22] D. P. Kingma、J. Ba，Adam: A Method for Stochastic Optimization. ICLR, 2015.

[23] I. Loshchilov、F. Hutter，Decoupled Weight Decay Regularization. ICLR, 2019.

[24] T. B. Brown 等，Language Models are Few-Shot Learners. NeurIPS, 2020.

[25] J. Kaplan 等，Scaling Laws for Neural Language Models. 2020.

[26] J. Hoffmann 等，Training Compute-Optimal Large Language Models. 2022.

[27] A. Holtzman、J. Buys、L. Du、M. Forbes、Y. Choi，The Curious Case of Neural Text Degeneration. ICLR, 2020.

[28] Y.-H. H. Tsai、S. Bai、M. Yamada、L.-P. Morency、R. Salakhutdinov，Transformer Dissection: An Unified Understanding for Transformer`s Attention via the Lens of Kernel. EMNLP-IJCNLP, K. Inui、J. Jiang、V. Ng、X. Wan 编，中国香港，Association for Computational Linguistics, 2019 年 11 月，4344–4353 页。https://doi.org/10.18653/v1/D19-1443

[29] A. Kazemnejad、I. Padhi、K. Natesan、P. Das、S. Reddy，The Impact of Positional Encoding on Length Generalization in Transformers. NeurIPS, 2023. https://openreview.net/forum?id=Drrl2gcjzl